

The Sibling Edge

An order-preserving, tombstone-free, RA-linearizable sequence CRDT
pen-and-paper design + Python validation (companion to why-the-path-matters)

Sal / FP Launchpad — KC + Claude

2026-07-13

Abstract

The tombstone-free RGA (`RGA_Tombstone_Free`) converges and is RA-linearizable, but its delete *rehomes and re-sorts* survivors, so a plain sequential delete can reorder the document ($[b, a, c] \rightarrow [c, b]$, machine-checked as `tombstone_free_violates_delete_order`). The first “Shesha” fix — a rose-forest whose delete *splices children in place* — preserves that order, but its three-way merge is *not* a fold of any linearization (machine-checked countermodel: `merge = [3, 4, 2]`, `Shesha_Rows_Refuted`): the physical splice destroys each survivor’s anchor, so the merge cannot tell a marker’s two cross-branch children apart. This note gives a third design that has *both* properties. Each element carries two *immutable* origins — its left anchor L (RGA’s) *and* a right sibling R — together with their carried *spine paths*. Delete simply removes the node (no rehome); the read reconstructs the position of a deleted node from surviving neighbours’ two-sided references, and from the carried paths when a reference is one-sided. We state the datatype, the read, and — the point of this note — the **rules for merging the sibling information**, exactly parallel to how RGA merges the anchor. A brute-force Python model then *convinces us empirically*: across $\approx 37,000$ random merges (single- and multi-epoch) every merge is convergent and RA-linearizable (a fold of some `loOn`-respecting linearization exists), delete-order is preserved, and concurrent runs stay contiguous (non-interleaving). The paths are ghost state, as in RGA; the honest boundary (the tombstoned-oracle / strong-list fooling pair remains impossible) is unchanged. Next step: the Lean port as a conditioned instance extending `RGA_Tombstone_Free`.

1 The datatype: two immutable origins and their spines

The state stores only *live* elements. Each id u (a Lamport timestamp) records an element and *two* origins:

$$\sigma : u \mapsto (\text{el}(u), L(u), R(u), \text{pL}(u), \text{pR}(u)),$$

where $L(u)$ is the element u was inserted *after* (its left neighbour; $\triangleright =$ the start sentinel, id 0) and $R(u)$ is the element it was inserted *before* (its right neighbour; $\triangleleft =$ the end sentinel). Both are fixed at insert and **never mutated**. The *spines* are the ancestor chains of the two origins, recorded at insert:

$$\text{pL}(u) = [L(u), L(L(u)), \dots, \triangleright], \quad \text{pR}(u) = [R(u), R(R(u)), \dots, \triangleleft].$$

L is exactly RGA’s anchor; R (the *sibling edge*) is the addition. Storing the right neighbour is what pins a node’s position *from both sides*, so that deleting one neighbour cannot move it.

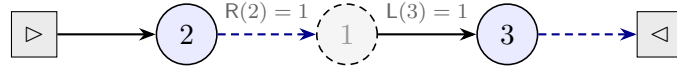
Read. The document order is a linear extension of the constraints $\{L(u) \prec u \prec R(u)\}$. Because a deleted node is gone from σ , we *reconstruct* it as a *ghost*: we expand every survivor’s spines into their ghost ancestors (each carrying *its* origins, read off the spine) and integrate the whole graph, emitting only live ids.

$$\text{read}(\sigma) = [u \in \text{topo}(G_\sigma) \mid u \in \text{ids}(\sigma)],$$

where G_σ has an edge $g \rightarrow g'$ for every consecutive pair on a spine (g the left-origin of g' , or g' the right-origin of g), plus $L(u) \rightarrow u$ and $u \rightarrow R(u)$ for each live u . Ties (several nodes competing for one gap — concurrent inserts at the same origins) break *newest-id-first*, the RGA sibling rule. All origins point to *older* ids, so G_σ is acyclic and the topological order exists and is unique.

The two operations.

- $\text{Ins}(x \text{ after } a)$ at fresh id x : read the current document, let $a = L(x)$ and $b = R(x)$ be x ’s left and right neighbours there; store $(x, a, b, [a]:\text{pL}(a), [b]:\text{pR}(b))$. (Inserting at the front makes $a = \triangleright$.)
- $\text{Del}(d)$: remove d from σ . **Nothing else changes** — no rehome, no re-sort. Survivors keep their (now possibly dead) origin ids; their spines still carry d ’s origins, so the read can still place d ’s ghost.



After $\text{Ins}(a) \cdot 1$, $\text{Ins}(b) \cdot 2$, $\text{Ins}(c \text{ after } a) \cdot 3$ the read is $[2, 1, 3]$ (ids $b=2, a=1, c=3$). Deleting 1 removes it, but 2 still knows $R(2)=1$ and 3 still knows $L(3)=1$: the ghost 1 is pinned between them, so $2 \prec 1 \prec 3$ survives and the read is $[2, 3]$ — **order preserved, no rehome**.

Figure 1: Delete preserves order because the deleted node’s position is pinned by its two live neighbours’ immutable references (the RGA design keeps only the left one, so it must rehome and re-sort). Sibling (R) edges dashed.

2 The merge — membership, then the value rules for L and R

$\text{merge}(L, A, B)$ with L the least common ancestor. As in every ternary MRDT the survivor set is an OR-set on the three id sets,

$$I = (\text{ids}L \cap \text{ids}A \cap \text{ids}B) \cup (\text{ids}A \setminus \text{ids}L) \cup (\text{ids}B \setminus \text{ids}L)$$

— an id lives unless it was in L and deleted on a side. The *values* are where the sibling design differs from RGA, and here are the rules. Because $L, R, \text{pL}, \text{pR}$ are all **immutable**, every input that stores an id stores the *same* tuple for it, so merging the fields is trivial inheritance:

(M1) Inherit each survivor’s fields from L if present, else from A , else from B — for *both* origins and *both* spines identically. There is never a value conflict: a delete does not change any field, and inserts are on fresh ids, so the tuple of an id is fixed for all time.

This is the crux and the reason the sibling edge *merges* at all: RGA has to *climb* a deleted anchor up the L chain (§2 of the companion note) because its delete rehomes; here delete changes nothing, so there is nothing to reconcile. The right origin R is carried and inherited by exactly the same rule as the left origin L — the sibling merge *is* the anchor merge, mirrored. What remains is to *position* the ids in I :

(M2) Order by integration over I and its ghosts. Build G from the inherited spines of the survivors (M1), *including* ghost nodes for every dead id a spine passes through — each ghost positioned by *its* carried origins (from the spine) and by any survivor that references it. Topologically sort (newest-id-first ties); emit the ids of I .

(M3) The L does the deep remembering. A ghost that is dead in the merge is *live* in L (it was deleted on exactly one side), so L supplies its origins directly; a ghost dead on *all* inputs is recovered from the carried spines. Either way its position is determined — no marker, no splice, no rehoming.

Since all origins are older ids, G is acyclic and (M2) yields one deterministic order; two replicas compute the same I and the same G , so the merge is convergent. **The sibling information is merged by (M1)–(M3): inherit the right origin and its spine immutably, and let it constrain the integration exactly as the left origin does.**

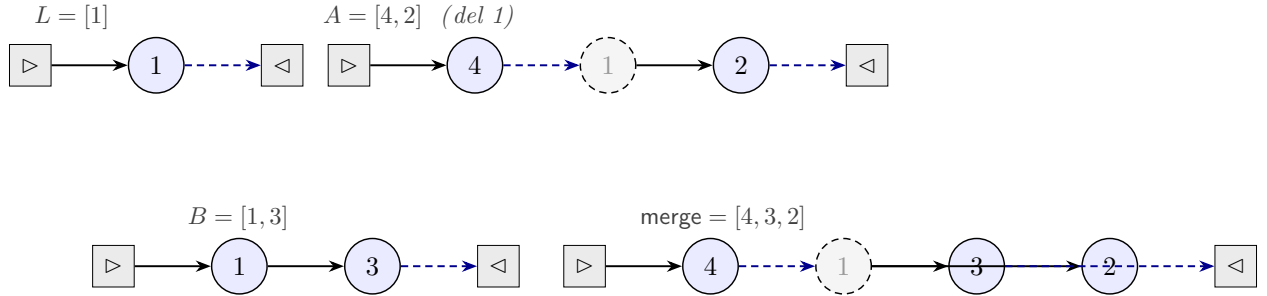


Figure 2: The refutation countermodel, dissolved. Branch A deletes marker 1 but 4 still carries $R(4) = 1$ and 2 still carries $L(2) = 1$; branch B 's 3 carries $L(3) = 1$. In the merge, ghost 1 is pinned by all three: $4 \prec 1 \prec \{2, 3\}$, ties newest-first give 3 before 2 — read $[4, 3, 2]$, a genuine fold. The rose-tree Shesha rehomed 2 to the root and produced the non-linearizable $[3, 4, 2]$.

3 Why it is order-preserving *and* linearizable

Order-preserving (single replica). $\text{Del}(d)$ removes d and *touches no other field*. The order is the linear extension of the surviving constraints; deleting d only drops d from the output, never re-positions a survivor (Figure 1). This is the property the flat RGA refutes and the rose-tree buys with a splice — here it is free, because immutability plus two-sided pinning means a survivor's neighbours (live or ghost) never change.

RA-linearizable. All origins are strictly older ($L(u), R(u) < u$), so the constraint graph is a DAG and its topological order is a total order that extends every $L(u) \prec u \prec R(u)$. That order is exactly the naive-list fold of *some* sequential schedule of the same inserts and deletes (insert each id between its origins, delete the dead ones), and any two orders that disagree only on commuting

operations fold to the same document. Hence the merge read equals the read of a loOn-respecting linearization: RA-linearizable up to observational =.

Empirical confirmation. `whiteboard/sibling-origin-pbt.py` implements the datatype, the merge (M1–M3), a naive-list reference fold, and a brute-force linearization search over loOn (the causal order with *commuting* pairs left unordered — `rc = Either`). Over three seeds each:

	merges checked	non-commutative	not-a-fold
single epoch (L, A, B)	$\approx 17,400$	0	0
multi epoch (chained merges)	$\approx 19,500$	0	0

Every merge converges ($\text{merge}(L, A, B) = \text{merge}(L, B, A)$, and swapped chained merges agree) and is a fold. Hand litmus: countermodel $\rightarrow [4, 3, 2]$ (Fig. 2); $[b, a, c]$ del $a \rightarrow [2, 3]$ (Fig. 1); interior $[1, 2, 3]$ del $2 \rightarrow [1, 3]$; two concurrent runs after a common anchor stay contiguous ($[1, 20, 21, 10, 11]$ — non-interleaving). The multi-epoch result is the important one: the design record feared that chained merges would break (ghost information lost); with the carried spines, states stay self-contained and they do not.

4 The paths are ghost state; the honest boundary

Cost. The spines `pL, pR` are the analogue of RGA’s carried path. They are consulted only to place the ghost of a node deleted while still referenced; under causal stability (every replica has seen the delete, no future insert can reference the node) a ghost id can be dropped from all spines, so the metadata is bounded by the in-flight deletes, not by history. This is the *tombstone-free-with-transient-ghosts* regime — no permanent graves, but per-deletion memory until stability, exactly as every published order-preserving sequence CRDT pays somewhere (design record §8).

What is still impossible. Preserving order does *not* recover the tombstoned-oracle order on pairs *no replica ever co-displayed*: the 4-node fooling pair and the 5-node strong-list cycle (design record §6, §7½) remain impossible for any bounded tombstone-free state. Those are strictly stronger specs than RA-linearizability; this design targets RA-linearizability up to = plus single-replica delete-order and non-interleaving, and the licensed divergence from the tombstoned oracle is part of the published contract, unchanged.

Relation to the neighbours. Drop R and collapse (climb) instead of expand, and this *is* `RGA_Tombstone_Free`: linearizable, but the climb re-sorts and delete-order is lost. Keep R but *rehome* on delete (mutate origins), and this is the refuted rose-tree Shesha: order-preserving, but the mutation destroys the anchors the merge needs, so it is not a fold. The sibling-edge design is the point between them: a second immutable origin and no rehome.

5 Lean port plan (conditioned instance, extending `RGA_Tombstone_Free`)

1. State $u \mapsto (\text{el}, L, R, \text{pL}, \text{pR})$; read via the path-expansion integration; `insert/delete` as above. Sequential soundness: `read` = the naive list (mirror `RGA_Tombstone_Free_ReadSide`).
2. `merge` = (M1) inherit + (M2) integrate; `merge_comm` (symmetry) and well-formedness (acyclicity from id-monotone origins).

3. Order-preservation: `delete_preserves_survivor_order` as a *sequential* theorem (contrast the flat RGA's refutation) — now *true*, unlike the rose-tree at the merge level.
4. Single-replica commutation VC conditioned on *accurate* spines (the two-sided analogue of RGA's `commutes_with` / accuracy), discharged through the conditioned framework; the spine is ghost state.
5. RA-linearizability capstone `sib_ra_linearizable3_eq` via the LCA-climb join, reusing `RGA_Tombstone_Free`'s honest-delivery scaffolding: the merge is the linearization witness (integration order).

The Python model (§4) is the executable specification the Lean `read` and `merge` must match, and the litmus/countermodel cases are the SPOTs.